

Security Remediation Report

Project-Ticket (Fute Portal). For Cybersecurity / Application Security review.

Prepared: 2026-08-24 **Source audit:** docs/PRODUCTION_READINESS_AUDIT.md (a 13-agent multi-pass review, 24 findings verified against the actual source code) **Scope:** main/backend (the Express server and Firestore database), main/frontend (the React website) **Verification method:** Static code verification only, meaning the code was checked route by route for correct HTTP status responses, matching imports and exports, and build checks. No live browser click-through was performed this session (see the Verification Limitations section).

1. Purpose

This report documents every fix applied in response to the production-readiness audit, organized by security relevance, so the Cybersecurity team can independently assess what changed, why it closes a real risk, and what still needs attention. It is written for review, not as a claim of a completed penetration test (a simulated attack used to test security).

2. Security and Session Fixes (Primary Interest)

2.1 Session revocation did not actually work

Finding IDs: F04 (HIGH), F06 (MEDIUM)

Problem: The frontend's network client had no way to react when the server said "your session is no longer valid" (an HTTP 401 response). An admin using the Security Center's "Force logout" or "Revoke session" action revoked the session on the server, but the victim's already-open browser tab kept working. The app never re-checked the login token, so a revoked or expired (7-day) login token stayed functionally "logged in" on that person's screen. Separately, clicking Logout only cleared the browser's local storage; there was no call to the server to actually invalidate the session record, so a copied token (from browser developer tools, a shared machine, or a synced backup) remained valid for up to 7 days after the user logged out.

Fix:

- `main/frontend/src/utils/api.js` : added logic so that any 401 response (except on the login/register pages themselves) clears local storage and redirects the user to the login page.
- `main/backend/controllers/authController.js` : a new `logout()` function now revokes the caller's own session record in the database, using the same session-revocation system the Security Center already relied on.

- `main/backend/middleware/authMiddleware.js` : the login token's session ID is now attached to the request so the logout function can identify exactly which session to revoke.
- `main/frontend/src/context/AuthContext.jsx` : logging out now calls the new logout endpoint on the server (best effort, meaning the local logout still happens even if that network call fails).

Result: An admin's "force logout" now takes effect on the next request from that session. A user's own Logout revokes the token on the server instead of only forgetting it locally.

2.2 Credentials file sitting in the project's working files

Finding IDs: F05 (HIGH), F09 (LOW)

Problem: `docs/login-credentials.pdf` , a file literally named for credentials, was not being ignored by version control (git) and was one `git add .` command away from being permanently saved into the project's history forever.

Fix: Added rules to the project's `.gitignore` file (a file that tells git which files to never track) matching `docs/*credential*` and `docs/*Credential*` . Confirmed, using the command `git check-ignore -v` , that the file is now excluded from being added.

Residual action for this team: The file itself was not deleted (this is not something to remove without an explicit instruction from the team). If it was ever committed on any branch in the past, treat every credential inside it as compromised and rotate (replace) them. A `.gitignore` rule only prevents future commits, it does not remove anything already in the project's history.

2.3 Unthrottled, unbounded expensive reads

Finding IDs: F03 (HIGH), F15 (MEDIUM), F17 (MEDIUM)

Problem: The default (no date-range filter) Analytics page, the Super Admin dashboard overview, and the SLA compliance view (SLA meaning Service Level Agreement, a target for how fast things should be resolved) each performed full, uncapped scans of the entire database with no rate limit (a cap on how often something can be requested) and no caching (temporarily saving a result so it doesn't need to be recalculated every time). These could be triggered repeatedly by any founder or superadmin account.

Fix:

- Every database scan in `analyticsController.js` , `dashboardController.js` , and `slaController.js` now carries an explicit limit on how many records it reads.
- 60-second (analytics) and 30-second (dashboard) temporary caches were added, matching the pattern already used elsewhere in the app.
- A shared rate limit (20 requests per minute) was added to the `/analytics` , `/analytics/export` , `/dashboard-overview` , and `/sla-compliance` routes in `founderRoutes.js` , as a backstop against someone varying their search parameters to get around the cache.

Result: Read cost and response time for these pages are now bounded and rate-limited instead of growing without limit alongside the total historical record count.

2.4 Missing rate limit on a secondary login-related endpoint

Finding ID: F08 (LOW)

Problem: `POST /api/auth/verify-password` (used to re-confirm a password before sensitive actions) had no rate limit, unlike the main login page. An attacker holding a stolen login token could try to guess the account's real password against this endpoint with no throttle stopping them.

Fix: The existing login rate limiter (10 attempts per 15 minutes, per IP address) is now applied to this route in `main/backend/routes/authRoutes.js`.

2.5 Authorization gap: organization-wide task read

Finding ID: F07 (LOW)

Problem: `GET /api/coordinator/tasks` had no restriction on the server side. Any logged-in employee account could read the entire organization's task backlog (including links to internal design and code files), not just their own assigned tasks. The website's "My Tasks" view was only filtering this down after already receiving everything.

Fix: `main/backend/controllers/taskProjectController.js`'s task-fetching function now adds a server-side filter for `assignee equals the requesting user's name` whenever the caller's role is "employee." Coordinators and Founders still keep the full, team-wide view they legitimately need.

2.6 Weak password floor on admin-issued resets

Finding ID: F10 (LOW)

Problem: Admin-issued password resets (the new password set right after an account gets locked) only required a 6-character minimum, the same weak floor as open self-registration, which undermined the protection the account lockout was meant to provide.

Fix: The minimum was raised to 10 characters, enforced both on the server (`superAdminController.js`) and reflected in the on-screen form (`SuperAdminUsersPage.jsx`).

2.7 Silent misconfiguration on missing secrets

Finding ID: F23 (MEDIUM)

Problem: `JWT_SECRET` (a secret key used to sign login tokens) was only checked for at the moment it was used, with no check when the server started up. A deployment missing this variable would start "successfully" and only fail the first time someone tried to log in, turning a configuration mistake into a live incident instead of a failed, easily-noticed deployment.

Fix: `main/backend/server.js` now checks for `process.env.JWT_SECRET` (the actual secret value) at startup and exits immediately with a clear error if it's missing, before the server starts accepting any traffic.

3. Reliability and Operational Hardening (Secondary Interest to Security)

ITEM	WHAT CHANGED
Health check (F24)	A new <code>GET /healthz</code> endpoint performs a real database read and reports whether it's reachable and how fast it responded, separate from the existing basic <code>GET /</code> check. This lets a monitoring tool tell "the process is running" apart from "the database is actually reachable."
Graceful shutdown (F24)	Standard shutdown signals now let any in-progress requests finish (using <code>server.close()</code>) before the server exits, instead of hard-killing active connections on a restart or redeploy.
Silent data loss on ticket edits (F20)	Editing a field that isn't actually editable used to appear to succeed on screen and then silently revert with no warning the next time the page refreshed. It is now checked and rejected with a visible notification before the screen ever shows the change.
Swallowed errors (F21)	<code>LeaveContext.applyLeave</code> now properly reports failures (matching every other place in the app that saves data), so a future part of the code can't accidentally show a false "success" message.
Truncated views with no signal (F16)	Founder-facing merged HR and IT views that hit their maximum record limit now include an <code>X-Results-Truncated</code> response marker instead of silently leaving records out with no indication anything was cut off.

4. Architecture Changes (No Behavior Change Intended)

Two large reorganizations were done purely for long-term maintainability. They're called out here specifically because reorganizing code like this is exactly the kind of change most likely to accidentally introduce a behavior change, which is what Section 6 below checks for.

CHANGE	DETAIL
Backend controller split (F01)	The 951-line, 31-function <code>founderController.js</code> file was split into 8 smaller files by topic: <code>superAdminController.js</code> , <code>analyticsController.js</code> , <code>slaController.js</code> , <code>notificationController.js</code> , <code>departmentController.js</code> , <code>permissionController.js</code> , <code>systemSettingsController.js</code> , <code>dashboardController.js</code> . All 31 original functions are preserved exactly as they were. Every route was individually re-checked after the split (see Section 6).
HR/IT controller de-duplication (F02)	The two nearly-identical complaint-handling controllers were combined into one shared <code>complaintControllerFactory.js</code> , used separately for HR and IT in <code>hrController.js</code> and <code>itController.js</code> . This means a bug fix now only needs to be made in one place instead of two.
Frontend component split (F11)	The 1,317-line <code>DashboardPage.jsx</code> file was broken down into <code>DataRequestsView.jsx</code> , <code>AssetsView.jsx</code> , <code>ReportsView.jsx</code> , and <code>RenderingStatusView.jsx</code> , with <code>DashboardPage.jsx</code> left as a thin file that just puts them together.
Shared pagination hook (F12)	A pattern for loading data page by page (cursor pagination), which was duplicated across three different parts of the frontend, was pulled out into one shared file: <code>hooks/useCursorPagination.js</code> .
Named constants (F13)	A hardcoded number, <code>.limit(200)</code> , that was repeated around 18 times across 9 different files, was replaced with named, reusable constants in <code>utils/constants.js</code> .

5. Deliberately Deferred (Not Fixed, But Tracked, Not Ignored)

FINDING	WHY DEFERRED
F14: Navigation permission list kept in sync by hand across layout files	Fixing this properly touches every role's navigation layout at the same time. Without a live browser available to verify the change this session, the risk of silently breaking navigation for one role outweighed the tidiness benefit. Left with its existing tracking comment in <code>PermissionsContext.jsx</code> .
F18: Data-loading logic starts fetching on login regardless of which page someone actually visits	This is already partially handled, since each part of the code checks the user's role and does nothing if it isn't relevant to them. A full fix would mean moving when each piece of data gets fetched from "as soon as you log in" to "only when you visit that specific page," across 6 different parts of the code and every page that uses them. This is documented as a known, deliberate tradeoff in <code>App.jsx</code> rather than rushed.
F22: The login form disables the browser's built-in form validation	The original audit itself rated this "not blocking."

6. Verification Method and Limitations

What was verified:

- Every backend route (31 founder/superadmin routes plus the new login and coordinator routes) was tested directly after every change. A broken or missing function would show up as a server error or crash; instead, everything correctly returned a 401 (meaning "you need to log in," which is the correct, expected response for a protected route being tested without logging in), not a 404 (not found) or 500 (server error).
- Every new or changed backend file's list of exported functions was compared against the original combined file's list, confirming an exact match with nothing accidentally dropped.
- Every new or changed frontend file was loaded through the website's development build process. A typing or syntax error there would return an error response; every single file loaded successfully instead.
- The backend process was confirmed to be running and responding correctly (via `/healthz` returning a success response) after every change.

What was not verified, and should be, before this ships:

- No live browser session was reachable from this working environment (the browser tool could reach the public internet but not the local development servers). **No actual login, then logout, then confirming the old token now fails, ticket creation, or clicking through the dashboard was performed.**
- The two largest reorganizations (Section 4) are exactly the kind of change most likely to hide a subtle bug (like a wrong setting passed to a component, or a missing function) behind a change that still technically "compiles." The static checks described above confirm "does it run," not "does it look and behave correctly on screen."

Recommended before production sign-off:

1. Log in, confirm the normal flow works, log out, and confirm the old login token now correctly fails instead of continuing to work.
2. Create one HR ticket and one IT ticket from start to finish.
3. Load the Super Admin's Analytics, Dashboard Overview, and SLA Compliance pages once each, and sanity-check the numbers shown.
4. Click "Load More" on the Tickets, Approvals, and Tasks pages once each.
5. Click through every IT dashboard tab (Dashboard, Tickets, Approval Center, Data Requests, Assets, Reports, Rendering Status).
6. Confirm `docs/login-credentials.pdf` was never previously committed to any branch. If it was, rotate (replace) every credential it contains.

7. File Inventory

CATEGORY	FILES
New backend files	<code>controllers/{superAdminUserController, analyticsController, slaController, notificationController, departmentController, permissionController, systemSettingsController, dashboardController, complaintControllerFactory}.js</code> , <code>utils/constants.js</code>
Removed	<code>controllers/founderController.js</code> (split into the 8 files above)
Modified backend files	<code>authController.js</code> , <code>hrController.js</code> , <code>itController.js</code> , <code>middleware/authMiddleware.js</code> , <code>routes/{authRoutes, founderRoutes}.js</code> , <code>server.js</code> , plus 6 controllers touched only to swap in the new named read-limit constant (<code>assetController</code> , <code>hrDeskController</code> , <code>leaveController</code> , <code>renderController</code> , <code>securityController</code> , <code>taskProjectController</code>)
New frontend files	<code>components/{DataRequestsView, AssetsView, ReportsView, RenderingStatusView}.jsx</code> , <code>hooks/useCursorPagination.js</code>
Modified frontend files	<code>utils/api.js</code> , <code>context/{AuthContext, ApprovalContext, LeaveContext, TaskProjectContext, TicketContext}.jsx</code> , <code>pages/{DashboardPage, SuperAdminUsersPage}.jsx</code> , <code>components/DataTable.tsx</code> , <code>App.jsx</code>
Config	<code>.gitignore</code>

Generated from `docs/PRODUCTION_READINESS_AUDIT.md` and the working session that implemented its findings. This document describes the change set as of 2026-08-24. It is not a substitute for an independent security review.